

ECE 315

Steven Knudsen, PhD, PEng

Lecture 13 :

- Hardware Abstraction Layer - HAL
- RP2040 UART Example



UNIVERSITY
OF ALBERTA



Last Lecture

- Serial Interfacing
- UART
- RS-232C, RS-422, RS-485

This Lecture

- Hardware Abstraction Layer – HAL
- RP2040 UART Example

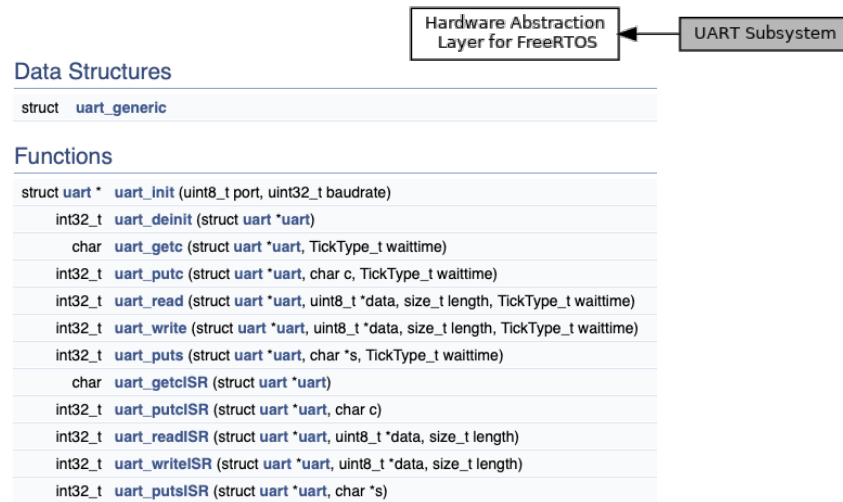
Learning Objectives

- Define “Hardware Abstraction Layer (HAL)” and why it’s needed
 - Understand the concept of a HAL as a software layer that separates application-level code from hardware-specific details
- Explain why a HAL is needed
 - Recognize how a HAL improves portability, maintainability, and code clarity by hiding processor-specific registers and configurations behind a consistent API
- Recognize and understand the structure and layered architecture of a HAL
 - Application layer
 - HAL (GPIO, UART, SPI/I2C, timers, ADC, DMA, PIO)
 - MCU port (register-level implementation)
 - BSP (board-specific pin maps, clocks, power setup)
 - Physical MCU hardware
 - And how each layer has clear responsibilities.

FreeRTOS UART Support

- Knowing how to use an UART is very useful in embedded programming
 - Not only for moving data between processors and peripherals
 - Also for debugging (simple command-line interface, debug “printf’s”, etc.)
- Up to the programmer to provide specialized functions to access the specific features of the target architecture (Zynq, RP2040, ...)
- FreeRTOS is, by definition, portable across processor architectures
 - In ECE 315 it’s used on the Zynq and RP2040 (Pico)
- For each processor, a kind of adaptation layer of software is needed

FreeRTOS UART Support – HAL



6

This is a project that defines common functionality like UART access based on an abstract architecture.

For each processor, a specific set of low-level functions is written that provide the functionality defined in the HAL.

FreeRTOS UART Support – HAL

- A now defunct Hardware Abstraction Layer (HAL) was proposed for FreeRTOS
- Each of the UART functions was defined without reference to specific UART functions on a processor
 - They were “abstracted” away from the underlying hardware
- Up to programmer to write the code to realize them



Hardware Abstraction Layer for FreeRTOS

UART Subsystem

Data Structures

```
struct uart_generic
```

Functions

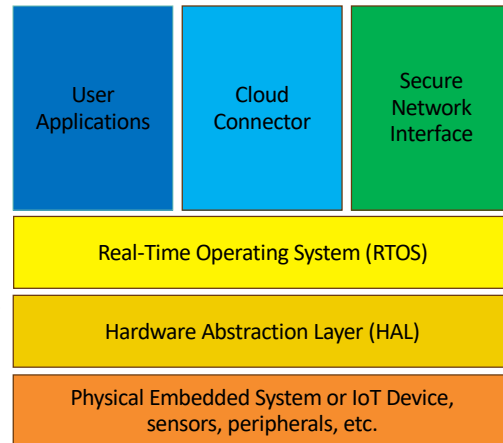
```
struct uart * uart_init (uint8_t port, uint32_t baudrate)
int32_t uart_deinit (struct uart *uart)
char uart_getc (struct uart *uart, TickType_t waittime)
int32_t uart_putc (struct uart *uart, char c, TickType_t waittime)
int32_t uart_read (struct uart *uart, uint8_t *data, size_t length, TickType_t waittime)
int32_t uart_write (struct uart *uart, uint8_t *data, size_t length, TickType_t waittime)
int32_t uart_puts (struct uart *uart, char *s, TickType_t waittime)
char uart_getcISR (struct uart *uart)
int32_t uart_putcISR (struct uart *uart, char c)
int32_t uart_readISR (struct uart *uart, uint8_t *data, size_t length)
int32_t uart_writeISR (struct uart *uart, uint8_t *data, size_t length)
int32_t uart_putsISR (struct uart *uart, char *s)
```

https://freertos-hal.github.io/doxygen/group_UART.html

7

What is a HAL?

- The HAL provides an abstraction layer to the processor and peripherals with hardware-specific configurations
- The RTOS provides thread and resource mgmt.
- The secure network interface provides an encrypted Internet connection using an appropriate interface (Ethernet, Wi-Fi, low-power radio).
- The cloud connector is a protocol interface to the cloud solution provider.
- The user application provides the bespoke functionality of the system.



An RP2040 HAL Example

HAL Layers

App (protocols, tasks)



HAL (gpio, uart, timer, pwm, spi/i2c, adc, dma, pio)



MCU Port (RP2040 regs, IRQ glue, clock tree, pad/io mux)



BSP (board pins, rails, clock & XOSC setup, pin functions)



RP2040 (dual M0+, SIO, IO_BANK0, TIMER, PWM, UART, I2C, SPI, PIO, DMA)

BSP

- Board Support Package
- Pin numbers, clock settings, etc.



Some Example Headers – GPIO

```
#include <stdint.h>
#include <stdbool.h>

typedef enum { GPIO_IN, GPIO_OUT } gpio_dir_t;
typedef enum { GPIO_PULL_NONE, GPIO_PULL_UP, GPIO_PULL_DOWN } gpio_pull_t;
typedef enum { GPIO_LOW=0, GPIO_HIGH=1 } gpio_level_t;

typedef struct { uint8_t pin; } gpio_t; // RP2040 uses global 0..29 numbering
typedef struct {
    gpio_dir_t dir;
    gpio_pull_t pull;
    bool open_drain; // emulated via OUT + OE control if needed
    uint8_t func; // 0=GPIO, else peripheral func (see BSP map)
    uint8_t drive_mA; // 2,4,8,12
    bool schmitt; // input schmitt trigger
    bool slew_fast; // fast slew
} gpio_cfg_t;
```

```
void gpio_init(const gpio_t *g, const gpio_cfg_t *cfg);
void gpio_write(const gpio_t *g, gpio_level_t v);
gpio_level_t gpio_read(const gpio_t *g);
void gpio_set(const gpio_t *g);
void gpio_clr(const gpio_t *g);
void gpio_toggle(const gpio_t *g);
```

Essential GPIO functions

GPIO Configuration struct



Made with help from ChatGPT 10

Some Example Headers – UART

```
#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
```

```
typedef enum { UART0=0, UART1=1 } uart_inst_t;
typedef enum { UART_OK, UART_EBUSY, UART_EPARM, UART_EIO, UART_ETIMEOUT } uart_status_t;
typedef enum { UART_PARITY_NONE, UART_PARITY_EVEN, UART_PARITY_ODD } uart_parity_t;
```

```
typedef struct {
    uart_inst_t inst;
    uint32_t baud;
    uint8_t data_bits; // 5..8
    uint8_t stop_bits; // 1 or 2
    uart_parity_t parity;
} uart_cfg_t;
```

```
typedef void (*uart_rx_cb_t)(uint8_t byte, void *ctx);

uart_status_t uart_init(const uart_cfg_t *cfg);
size_t uart_write(uart_inst_t inst, const uint8_t *buf, size_t len, uint32_t timeout_ms);
size_t uart_read (uart_inst_t inst, uint8_t *buf, size_t len, uint32_t timeout_ms);
uart_status_t uart_set_rx_callback(uart_inst_t inst, uart_rx_cb_t cb, void *ctx); // IRQ-driven
```

Essential UART functions

UART Configuration struct



11

Some Example Headers and Code – BSP

```
#include "hal/gpio.h"
#include "hal/pwm.h"

extern const gpio_t LED;      // e.g., GPIO25 on Pico
extern const gpio_t UART0_TX; // your routing choice
extern const gpio_t UART0_RX;

void board_clock_init(void); // XOSC + PLLs → clk_sys, clk_peri stable
void board_pinmux_init(void); // IO_BANK0_FUNCSEL, pulls, drive strength
void board_power_init(void);  // (usually NOP on Pico-class boards)
```

Board Support Package header file
bsp/board.h

Board Support Package c file
bsp/board.c



UNIVERSITY
OF ALBERTA

12

```
#include "board.h"

const gpio_t LED      = { .pin = 25 };
const gpio_t UART0_TX = { .pin = 0  }; // GP0
const gpio_t UART0_RX = { .pin = 1  }; // GP1

void board_clock_init(void) {
    // 1) Enable XOSC to 12 MHz
    // 2) Configure PLLs (SYS ~125 MHz, USB 48 MHz), switch clk_sys
    // 3) Gate clk_peri = clk_sys for UART/SPI/I2C
    // (Done with raw regs in mcu/rp2040/clock.c so HAL stays clean)
}

void board_pinmux_init(void) {
    // Map UART0 to GP0/GP1, LED to GPIO function, etc.
    // Configure pads: pulls, drive, schmitt based on desired defaults.
}

void board_power_init(void) { /* usually nothing for RP2040 dev boards */ }
```

We have to fill in the blanks here, in board.c

MCU port (register-level): RP2040-specific

```
#include "hal/gpio.h"
#include "mcu/rp2040/regs.h" // your local header or Pico SDK

static inline void pad_cfg(uint8_t pin, const gpio_cfg_t *cfg) {
    // PADS_BANK0: pullups/pulldowns, drive, schmitt, slew
    volatile padsbank0_hw_t *P = PADS_BANK0;

    uint32_t v = 0;
    if (cfg->pull == GPIO_PULL_UP)    v |= PADS_BANK0_GPIO_PUE_BITS;
    if (cfg->pull == GPIO_PULL_DOWN) v |= PADS_BANK0_GPIO_PDE_BITS;
    if (cfg->slew_fast)                v |= PADS_BANK0_GPIO_SLEWFAST_BITS;
    if (cfg->schmitt)                  v |= PADS_BANK0_GPIO_SCHMITT_BITS;
    // drive_mA → DRV field (00=2mA,01=4mA,10=8mA,11=12mA)
    v |= ((cfg->drive_ma >= 12) ? 3 : (cfg->drive_ma >= 8) ? 2 : (cfg->drive_ma >= 4) ? 1 : 0)
        << PADS_BANK0_GPIO_DRV_LSB;
    P->io[pin] = v;
}
```

GPIO support
mcu/rp2040/gpio_rp2040.c

MCU port (register-level): RP2040-specific

```
static inline void funcsel(uint8_t pin, uint8_t func) {  
    // IO_BANK0: FUNCSEL chooses peripheral (0=SPI0,1=UART0,2=I2C0,...,31=GPIO)  
    iobank0_hw_t *I = IO_BANK0;  
    uint32_t ctrl = I->io[pin].ctrl;  
    ctrl &= ~IO_BANK0_GPIO0_CTRL_FUNCSEL_BITS;  
    ctrl |= ((uint32_t)func << IO_BANK0_GPIO0_CTRL_FUNCSEL_LSB);  
    I->io[pin].ctrl = ctrl;  
}
```

```
void gpio_init(const gpio_t *g, const gpio_cfg_t *cfg) {  
    // Ensure peripheral is out of reset and clk_peri enabled done by board_clock_init  
    pad_cfg(g->pin, cfg);  
    funcsel(g->pin, cfg->func ? cfg->func : 31 /*GPIO*/);  
  
    // SIO direction & initial level  
    if (cfg->dir == GPIO_OUT) {  
        SIO->gpio_oe_set = (1u << g->pin);  
    } else {  
        SIO->gpio_oe_clr = (1u << g->pin);  
    }  
}
```

MCU port (register-level): RP2040-specific

```
void gpio_write(const gpio_t *g, gpio_level_t v) {
    if (v) SIO->gpio_set = (1u << g->pin);
    else   SIO->gpio_clr = (1u << g->pin);
}

gpio_level_t gpio_read(const gpio_t *g) {
    return (SIO->gpio_in & (1u << g->pin)) ? GPIO_HIGH : GPIO_LOW;
}

void gpio_set(const gpio_t *g) { SIO->gpio_set = (1u << g->pin); }
void gpio_clr(const gpio_t *g) { SIO->gpio_clr = (1u << g->pin); }
void gpio_toggle(const gpio_t *g){ SIO->gpio_xor = (1u << g->pin); }
```

MCU port (register-level): RP2040-specific

```
#include "hal/uart.h"
#include "mcu/rp2040/regs.h"

typedef struct { uart_rx_cb_t cb; void *ctx; } rx_ctx_t;
static rx_ctx_t s_rx[2];

static inline uart_hw_t* U(uart_inst_t i){ return (i==UART0) ? UART0_HW : UART1_HW; }
```

GPIO support
mcu/rp2040/uart_rp2040.c

MCU port (register-level): RP2040-specific

```
uart_status_t uart_init(const uart_cfg_t *cfg) {
    uart_hw_t *u = U(cfg->inst);
    // Disable before config
    u->cr = 0;

    // Baud: IBRD/FBRD from clk_peri (board set)
    uint32_t baud = cfg->baud;
    // (provide via your regs shim or store during clock init)
    uint32_t clk = CLOCKS->clk_peri_ctrl_hz;
    uint32_t bauddiv_x64 = (clk << 6) / (16 * baud);
    u->ibrd = bauddiv_x64 >> 6;
    u->fbrd = bauddiv_x64 & 0x3F;

    // LCR: data bits, parity, stop
    uint32_t lcr = 0;
    lcr |= (cfg->data_bits - 5) << UART_LCR_H_WLEN_LSB; // 5..8 → 0..3
    if (cfg->stop_bits == 2) lcr |= UART_LCR_H_STP2_BITS;
    if (cfg->parity != UART_PARITY_NONE) {
        lcr |= UART_LCR_H_PEN_BITS;
        if (cfg->parity == UART_PARITY_EVEN) lcr |= UART_LCR_H_EPS_BITS;
    }
    u->lcr_h = lcr;

    // Enable TX/RX
    u->cr = UART_CR_TXE_BITS | UART_CR_RXE_BITS | UART_CR_UARTEN_BITS;
    return UART_OK;
}
```

MCU port (register-level): RP2040-specific

```
size_t uart_write(uart_inst_t inst, const uint8_t *buf, size_t len, uint32_t timeout_ms) {
    uart_hw_t *u = U(inst);
    uint64_t deadline = timer_now_us() + (uint64_t)timeout_ms * 1000;
    size_t n=0;
    while (n < len) {
        if (!(u->fr & UART_FR_TXFF_BITS)) { u->dr = buf[n++]; }
        else if (timer_now_us() > deadline) break;
    }
    return n;
}
```

```
size_t uart_read(uart_inst_t inst, uint8_t *buf, size_t len, uint32_t timeout_ms) {
    uart_hw_t *u = U(inst);
    uint64_t deadline = timer_now_us() + (uint64_t)timeout_ms * 1000;
    size_t n=0;
    while (n < len) {
        if (!(u->fr & UART_FR_RXFE_BITS)) { buf[n++] = (uint8_t)u->dr; }
        else if (timer_now_us() > deadline) break;
    }
    return n;
}
```



MCU port (register-level): RP2040-specific

```
uart_status_t uart_set_rx_callback(uart_inst_t inst, uart_rx_cb_t cb, void *ctx) {  
    s_rx[inst] = (rx_ctx_t){cb, ctx};  
    U(inst)->imsc |= UART_IMSC_RXIM_BITS; // enable RX interrupt  
    // Enable NVIC IRQ for UART0/1 in your vector table  
    return UART_OK;  
}
```

```
// ISR  
void UART0_IRQ_Handler(void) {  
    uint32_t mis = UART0_HW->mis;  
    if (mis & UART_MIS_RXMIS_BITS) {  
        while (!(UART0_HW->fr & UART_FR_RXFE_BITS)) {  
            uint8_t b = (uint8_t)UART0_HW->dr;  
            if (s_rx[0].cb) s_rx[0].cb(b, s_rx[0].ctx);  
        }  
        UART0_HW->icr = UART_ICR_RXIC_BITS;  
    }  
}
```

RP2040 App using the HAL

```
#include "bsp/board.h"
#include "hal/gpio.h"
#include "hal/uart.h"
#include "hal/timer.h"
```

```
static void on_tick(void *ctx) {
    (void)ctx;
    extern const gpio_t LED;
    gpio_toggle(&LED);
}
```

```
static void on_uart(uint8_t b, void *ctx) {
    (void)ctx;
    const uint8_t e[2] = { b, '\n' };
    uart_write(UART0, e, 2, 10);
}
```



UNIVERSITY
OF ALBERTA

```
int main(void) {
    board_power_init();
    board_clock_init();
    board_pinmux_init();
    timer_init();

    gpio_cfg_t led_cfg = {.dir=GPIO_OUT, .pull=GPIO_PULL_NONE, .func=31,
                          .drive_mA=8, .schmitt=true};
    gpio_init(&LED, &led_cfg);

    uart_cfg_t u0 = { .inst=UART0, .baud=115200, .data_bits=8, .stop_bits=1,
                     .parity=UART_PARITY_NONE };
    uart_init(&u0);
    uart_set_rx_callback(UART0, on_uart, NULL);

    timer_start(/*alarm*/0, /*period*/250000, TMR_PERIODIC, on_tick, NULL);

    while (1) { __wfi(); } // sleep until IRQ
}
```

20

RP2040 App using the HAL

```
#include "bsp/board.h"
#include "hal/gpio.h"
#include "hal/uart.h"
#include "hal/timer.h"
```

```
static void on_tick(void *ctx) {
    (void)ctx;
    extern const gpio_t LED;
    gpio_toggle(&LED);
}
```

```
static void on_uart(uint8_t b, void *ctx) {
    (void)ctx;
    const uint8_t e[2] = { b, '\n' };
    uart_write(UART0, e, 2, 10);
}
```

```
int main(void) {
    board_power_init();
    board_clock_init();
    board_pinmux_init();
    timer_init();

    gpio_cfg_t led_cfg = {.dir=GPIO_OUT, .pull=GPIO_PULL_NONE, .func=31,
                          .drive_mA=8, .schmitt=true};
    gpio_init(&LED, &led_cfg);

    uart_cfg_t u0 = { .inst=UART0, .baud=115200, .data_bits=8, .stop_bits=1,
                     .parity=UART_PARITY_NONE };
    uart_init(&u0);
    uart_set_rx_callback(UART0, on_uart, NULL);

    timer_start(/*alarm*/0, /*period*/250000, TMR_PERIODIC, on_tick, NULL);

    while (1) { __wfi(); } // sleep until IRQ
}
```



RP2040 App using the HAL

```
#include "bsp/board.h"
#include "hal/gpio.h"
#include "hal/uart.h"
#include "hal/timer.h"
```

```
static void on_tick(void *ctx) {
    (void)ctx;
    extern const gpio_t LED;
    gpio_toggle(&LED);
}
```

```
static void on_uart(uint8_t b, void *ctx) {
    (void)ctx;
    const uint8_t e[2] = { b, '\n' };
    uart_write(UART0, e, 2, 10);
}
```

```
int main(void) {
    board_power_init();
    board_clock_init();
    board_pinmux_init();
    timer_init();

    gpio_cfg_t led_cfg = {.dir=GPIO_OUT, .pull=GPIO_PULL_NONE, .func=31,
                          .drive_mA=8, .schmitt=true};
    gpio_init(&LED, &led_cfg);

    uart_cfg_t u0 = { .inst=UART0, .baud=115200, .data_bits=8, .stop_bits=1,
                     .parity=UART_PARITY_NONE };
    uart_init(&u0);
    uart_set_rx_callback(UART0, on_uart, NULL);

    timer_start(/*alarm*/0, /*period*/250000, TMR_PERIODIC, on_tick, NULL);

    while (1) { __wfi(); } // sleep until IRQ
}
```



Zynq UART for FreeRTOS

- In ECE 315 labs the Zybo (Zynq) UART is used
- The API from Xilinx, `uartps`, is used within FreeRTOS-based functions to manage UART transmit and receive
- For example, let's look at some of the UART code from the lab...

```
void interruptHandler(void *CallBackRef, u32 event, unsigned int
EventData)
{
    if (event == RECEIVED_DATA){
        handleReceiveEvent();
    } else if (event == SENT_DATA){
        handleSentEvent();
    } else {
        xil_printf("Neither a RECEIVE event nor a SEND event\n");
    }
}
```

Zynq UART for FreeRTOS

```
void interruptHandler(void *CallBackRef, u32 event, unsigned int EventData)
{
    if (event == RECEIVED_DATA) {
        handleReceiveEvent();
    } else if (event == SENT_DATA) {
        handleSentEvent();
    } else {
        xil_printf("Neither a RECEIVE event nor a SEND event\n");
    }
}
```

Annotations in the code block:

- Two orange arrows point to `handleReceiveEvent();` and `handleSentEvent();`.
- A pink box labeled "Xilinx API in purple" points to the `xil_printf` function call.

```

void interruptHandler(void *CallBackRef, u32 event, unsigned
int EventData)
{
    if (event == RECEIVED_DATA){
        handleReceiveEvent();
    } else if (event == SENT_DATA){
        handleSentEvent();
    } else {
        xil_printf("Neither a RECEIVE event nor a SEND event\n");
    }
}

```

```

void handleReceiveEvent()
{
    u8 receive_buffer;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    countRxIrq++;
    while (XUartPs_IsReceiveData(UART_BASEADDR)){
        receive_buffer = XUartPs_ReadReg( UART_BASEADDR, UART_FIFO_OFFSET);
        xQueueSendFromISR( xRxQueue, &receive_buffer, &xHigherPriorityTaskWoken);
    }
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}

void handleSentEvent()
{
    u8 transmit_data;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    countTxIrq++;
    transmitDataFromQueue(&transmit_data, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    disableTxEmpty();
}

void transmitDataFromQueue(u8 *data, BaseType_t *taskToSwitch)
{
    if (XUartPs_IsTransmitEmpty(&UART)){
        while (uxQueueMessagesWaitingFromISR(xTxQueue) > 0 ){
            /***** Enter your code here *****/
            // TODO: receive data from the queue and send it through the UART interface
            /*****
        }
    }
}

```

25

Zynq UART for FreeRTOS



Zynq UART API and FreeRTOS HAL

Data Structures

struct **uart_generic**

Functions

struct uart *	uart_init (uint8_t port, uint32_t baudrate)
int32_t	uart_deinit (struct uart *uart)
char	uart_getc (struct uart *uart, TickType_t waittime)
int32_t	uart_putc (struct uart *uart, char c, TickType_t waittime)
int32_t	uart_read (struct uart *uart, uint8_t *data, size_t length, TickType_t waittime)
int32_t	uart_write (struct uart *uart, uint8_t *data, size_t length, TickType_t waittime)
int32_t	uart_puts (struct uart *uart, char *s, TickType_t waittime)
char	uart_getcISR (struct uart *uart)
int32_t	uart_putcISR (struct uart *uart, char c)
int32_t	uart_readISR (struct uart *uart, uint8_t *data, size_t length)
int32_t	uart_writeISR (struct uart *uart, uint8_t *data, size_t length)
int32_t	uart_putsISR (struct uart *uart, char *s)

https://freertos-hal.github.io/doxygen/group__UART.html

https://xilinx.github.io/embeddedsw.github.io/uartps/doc/html/api/uartps_8h.html



Functions

XUartPs_Config *	XUartPs_LookupConfig (u16 DeviceId) Looks up the device configuration based on the unique device ID. More...
s32	XUartPs_CfgInitialize (XUartPs *InstancePtr, XUartPs_Config *Config, u32 EffectiveAddr) Initializes a specific XUartPs instance such that it is ready to be used. More...
u32	XUartPs_Send (XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes) This function sends the specified buffer using the device in either polled or interrupt driven mode. More...
u32	XUartPs_Recv (XUartPs *InstancePtr, u8 *BufferPtr, u32 NumBytes) This function attempts to receive a specified number of bytes of data from the device and store it into the sp
s32	XUartPs_SetBaudRate (XUartPs *InstancePtr, u32 BaudRate) Sets the baud rate for the device. More...
void	XUartPs_SetOptions (XUartPs *InstancePtr, u16 Options) Sets the options for the specified driver instance. More...
u16	XUartPs_GetOptions (XUartPs *InstancePtr) Gets the options for the specified driver instance. More...
void	XUartPs_SetFifoThreshold (XUartPs *InstancePtr, u8 TriggerLevel) This function sets the receive FIFO trigger level. More...
u8	XUartPs_GetFifoThreshold (XUartPs *InstancePtr) This function gets the receive FIFO trigger level. More...
u16	XUartPs_GetModemStatus (XUartPs *InstancePtr) This function gets the modem status from the specified UART. More...
u32	XUartPs_IsSending (XUartPs *InstancePtr) This function determines if the specified UART is sending data. More...
u8	XUartPs_GetOperMode (XUartPs *InstancePtr) This function gets the operational mode of the UART. More...
void	XUartPs_SetOperMode (XUartPs *InstancePtr, u8 OperationMode) This function sets the operational mode of the UART. More...
u8	XUartPs_GetFlowDelay (XUartPs *InstancePtr) This function sets the Flow Delay. More...
void	XUartPs_SetFlowDelay (XUartPs *InstancePtr, u8 FlowDelayValue) This function sets the Flow Delay. More...
u8	XUartPs_GetRecvTimeout (XUartPs *InstancePtr) This function gets the Receive Timeout of the UART. More...
void	XUartPs_SetRecvTimeout (XUartPs *InstancePtr, u8 RecvTimeout) This function sets the Receive Timeout of the UART. More...
s32	XUartPs_SetDataFormat (XUartPs *InstancePtr, XUartPsFormat *FormatPtr) Sets the data format for the device. More...
void	XUartPs_GetDataFormat (XUartPs *InstancePtr, XUartPsFormat *FormatPtr) Gets the data format for the specified UART. More...
u32	XUartPs_GetInterruptMask (XUartPs *InstancePtr) This function gets the interrupt mask. More...
void	XUartPs_SetInterruptMask (XUartPs *InstancePtr, u32 Mask) This function sets the interrupt mask. More...
void	XUartPs_InterruptHandler (XUartPs *InstancePtr) This function is the interrupt handler for the driver. More...

26

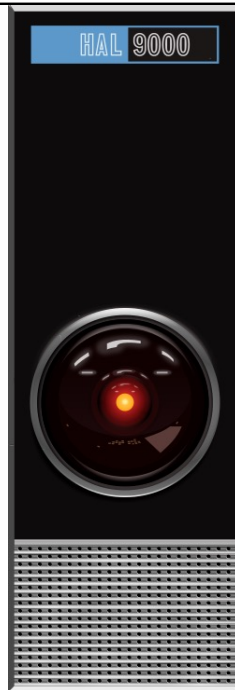
Can see how the Xilinx API might be used to

This would allow us to have the same FreeR only need to have the Xilinx version to compile into the supporting code.

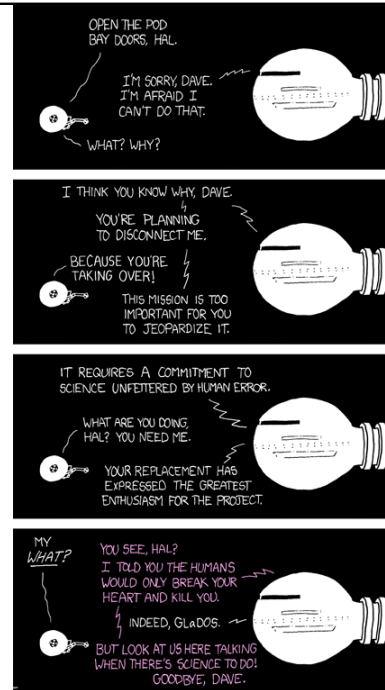
Next Lecture

- More Serial...
- Ethernet LAN
- CAN
- USB

Questions



https://en.wikipedia.org/w/index.php?title=HAL_9000&oldid=1311674673



<https://xkcd.com/375/>